Google Cloud H2S

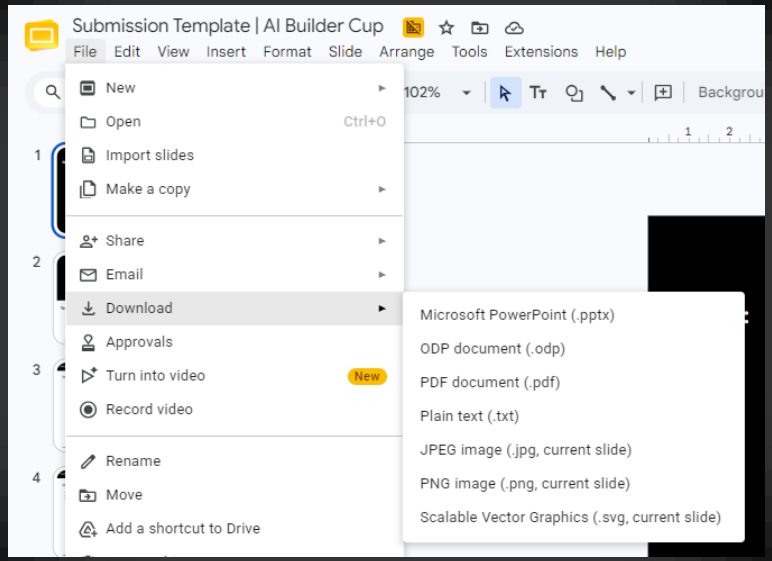
AI Builder Cup

Build in JAPAC.
Build for the world.



## Team Details

a. Team name: gTaprobane
b. Team leader name: Yogendran Nadaraja
c. Problem Statement: A supermarket card's headline "% off" rarely covers the whole basket, so Sri Lankan shoppers cannot tell what an offer truly saves after its exclusions, minimum bill, and cap.


Submission Template | AI Builder Cup
File
Edit
View
Insert
Format
Slide
Arrange
Tools
Extensions
Help
New
Open
Ctrl+O
Import slides
Make a copy
Share
Email
Download
Approvals
Turn into video
New
Record video
Rename
Move
Add a shortcut to Drive
Microsoft PowerPoint (.pptx)
ODP document (.odp)
PDF document (.pdf)
Plain text (.txt)
JPEG image (.jpg, current slide)
PNG image (.png, current slide)
Scalable Vector Graphics (.svg, current slide)

## Brief about the idea

Savvy Cart AI is a condition-aware grocery basket optimizer for Sri Lankan households. Gemini reads a shopper's receipts and bank offers; deterministic code then plans the cheapest **legal** way to buy what they need on a chosen day — across up to four stores, one card per bill — applying every exclusion, minimum bill, cap, and store or card rule in code, never guessing with a model.

Its **Offer Reality Check** turns a pasted bank offer into what the headline promises versus what the terms actually pay on this shopper's basket. The app is live on Cloud Run, built with Gemini and the Agent Development Kit: a guest, no-login, five-step flow that ends in a MOCK approval — no order is ever sent.

## Opportunities

- How different is it from any of the other existing ideas?
- How will it be able to solve the problem?
- USP of the proposed solution

• **How different:** price-comparison apps find the cheapest store per item. Savvy Cart prices the whole basket after card-offer exclusions, minimums, caps, and cannot-combine rules — and shows, line by line, when a famous “25% off” actually pays LKR 0.

• **How it solves the problem:** Gemini turns receipts and offer text into structured data; an exact OR-Tools optimizer assigns each item to a store and applies every rule; the shopper compares baseline vs optimized, sees the best day, and approves. Nothing is auto-ordered.

• **USP:** the model reads documents; deterministic code owns every rupee, so the plan is auditable, not a guess. Two grounding layers stop the model inventing a price, discount, or total.

## List of features offered by the solution

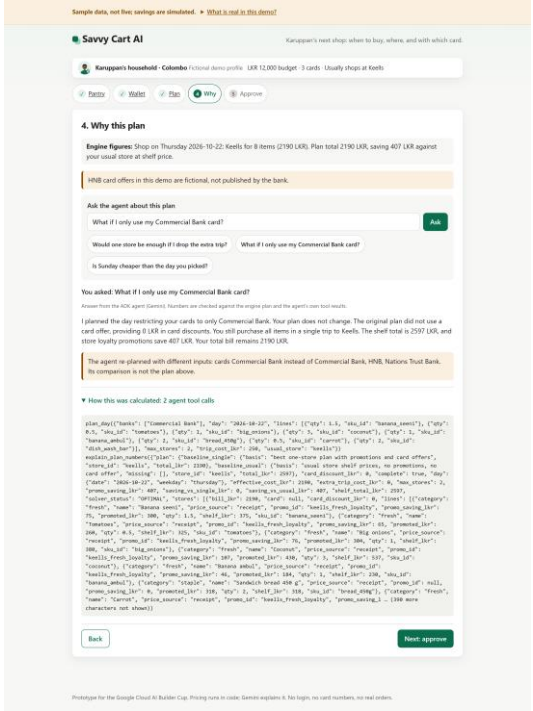
- Receipt-driven pantry — due dates from real purchase history (median gap), not a forecasting model.
- Gemini receipt extraction — a photo becomes catalog-matched lines, each with a confidence score.
- Offer Reality Check — Gemini reads a bank offer's text or screenshot into validated, editable terms.
- Exact multi-store optimizer (OR-Tools CP-SAT) — up to four stores, store promotions, one card offer per bill, and the cost of an extra trip.
- Ask the agent (ADK) — natural-language questions, on-demand re-planning, and a visible tool trace.
- Honest by design — it says "this offer pays LKR 0", invents no numbers, previews a MOCK order, and never touches a card number.

## Process flow diagram or Use-case diagram

A guest opens the Cloud Run URL — no login — and moves through a five-step wizard:

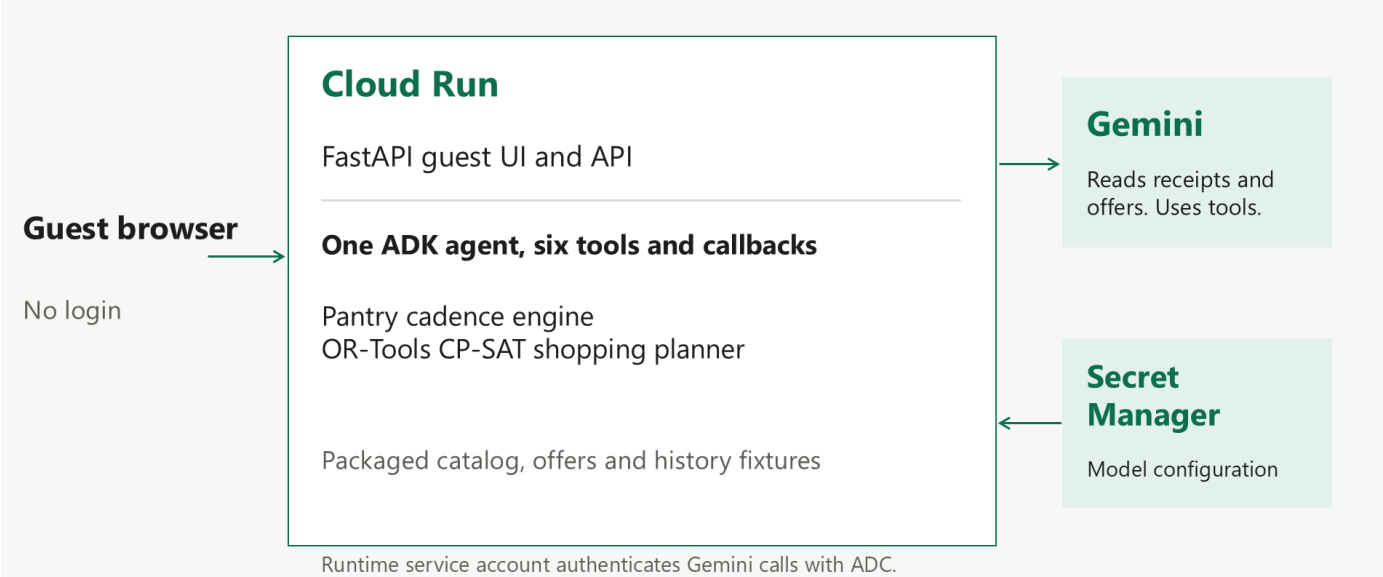
- **1 Pantry —** receipt history shows what is running low; add a receipt (the sample, or a photo read by Gemini).
- **2 Wallet —** choose cards, stores, and trip cost; run Offer Reality Check on any bank offer.
- **3 Plan —** the OR-Tools optimizer picks the stores, store promotions, best card offer, and best day.
- **4 Why —** the ADK agent explains the plan, re-plans on request, and shows every tool call it made.
- **5 Approve —** a MOCK order preview. No order is sent.

# Wireframes/Mock diagrams of the proposed solution (optional)




The deployed guest UI is the working prototype — these are live screens, not mockups. Shown: the **Why** step, where the ADK agent answers the shopper's question in plain language and reveals every tool call it made (plan_day, explain_plan_numbers). Every figure is checked against the engine before it appears.

# Architecture diagram of the proposed solution




Cloud Run (FastAPI, scale-to-zero) hosts the guest UI, the one ADK agent, and the OR-Tools engine in a single container. Gemini (gemini-3.8-flash on the Agent Platform) reads receipts and offers and explains plans. Secret Manager holds the model id; the runtime uses keyless Application Default Credentials. Data is packaged fixtures.

## Technologies to be used in the solution

- **Google Cloud —** Cloud Run (asia-southeast1, scale-to-zero); Gemini gemini-3.8-flash on the Gemini Enterprise Agent Platform; Secret Manager.
- **GenAI & agents —** Agent Development Kit (google-adk 2.x) and google-genai.
- **Optimization —** Google OR-Tools (CP-SAT) exact solver.
- **Application —** Python 3.12, FastAPI, uvicorn; server-rendered HTML with a lightweight vanilla JS/CSS frontend; httpx.
- **Build & test —** Docker (python:3.12-slim, non-root); pytest — 120 tests passing.
- **Not used in the prototype —** Firebase and Firestore; the demo runs on committed fixtures, with Firestore as the documented path to production scale.

# Estimated implementation cost (optional)

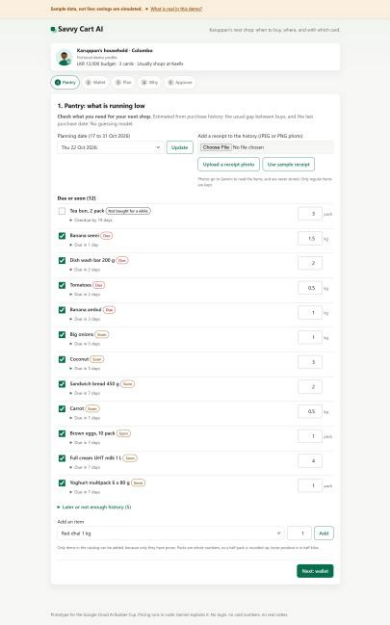
**Prototype (evaluation window):**

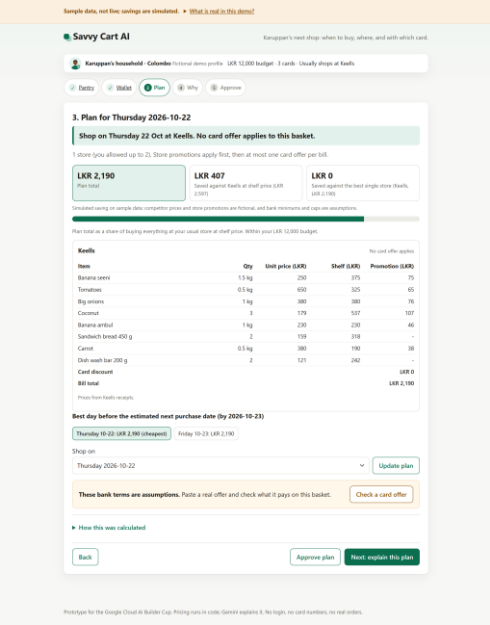
- Cloud Run scales to zero (min 0, max 3) — no paid warm instances.
- Gemini Flash traffic is judge-only; no managed-database cost (committed fixtures).
- Estimated spend: single-digit to low-tens of USD, typically within trial credits.

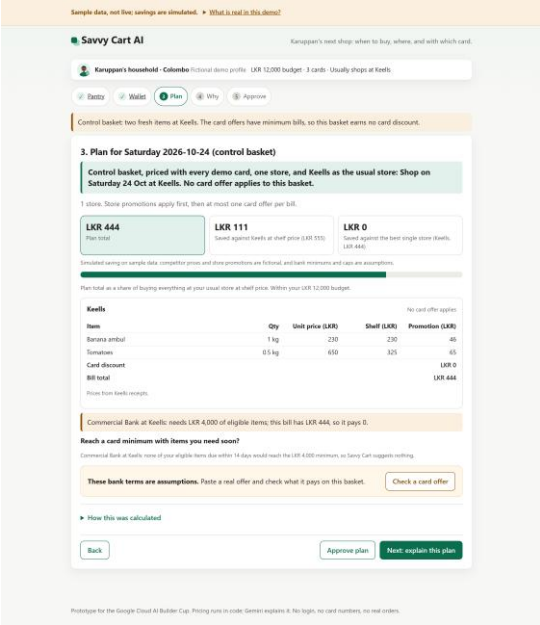
**At production scale (indicative):**

- Cost tracks active households: roughly one Gemini vision call and one explanation per planning session, plus scale-to-zero Cloud Run.
- Add Firestore (sessions, catalog/offer snapshots) and Cloud Storage (receipts): low-tens to low-hundreds of USD per month for an early pilot, before any retailer-data licensing.

# Snapshots of the prototype




Live screens: the receipt-driven pantry; the optimized plan line by line; and the honest case where a card offer's minimum is not met and it pays LKR 0. Sample data — savings are simulated.

## Prototype Performance report/Benchmarking

- 120 automated tests pass — covering deterministic pricing, offer eligibility, and the grounding guardrails.
- Live Gemini (checked 18 Oct 2026): the agent's Why answer about 30 seconds; reading a bank offer about 7 seconds.
- Worked example (simulated): a default basket costs LKR 4,604 across Keells and Cargills — LKR 1,328 below buying everything at Keells shelf price. The two-store split beats the best single store by only LKR 63, and at a LKR 320 trip cost the optimizer drops to one store.
- Offer Reality Check: a "10% off Saturday" headline worth LKR 593 on paper pays LKR 0 — the basket is LKR 2,475 short of the LKR 8,000 minimum.
- Every saving is simulated on labeled sample data.

## Additional Details/Future Development (if any)

- **Roadmap —** a Colombo household pilot; then authorized retailer and issuer data feeds (versioned prices, real card terms); then adapting the rule engine to other JAPAC markets (currencies, rounding, local card offers).
- **Business model —** free for households; later, issuers or retailers pay for consented, disclosed referrals. We never sell the shopper's data.
- **Beyond the prototype —** live prices, real orders and payments, Firestore persistence, and partner checkout are all post-Cup.

Provide links to your:

- GitHub Public Repository https://github.com/nyogendran/savvy-cart-ai
- Demo Video Link (3 Minutes) (to be added before submission)
- Final Product Link https://savvy-cart-890425261137.asia-southeast1.run.app

Google Cloud H2S

# AI Builder Cup

Build in JAPAC.
Build for the world.

# Thank you

